Exoplanet Habitability Catalog – Comprehensive Technical Report

1. Executive Summary

The **Exoplanet Habitability Catalog** is an end-to-end Database Management System (DBMS) application. It ingests astronomical data, applies a Machine Learning model to predict planetary habitability, stores the results in a relational database, and exposes this data through a modern web interface.

This report provides a line-by-line explanation of the codebase, the underlying theoretical concepts, and the exact execution flow.

2. Core DBMS Concepts & Database Schema

The database is built on **MySQL**, a Relational Database Management System (RDBMS). We heavily utilize the concept of **Data Normalization** (specifically 3rd Normal Form) to eliminate data redundancy. Instead of putting star data and planet data in one giant table, we split them into `stars` and `exoplanets`.

The Tables (`schema.sql`)

```
CREATE TABLE stars (
star_id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
temperature DECIMAL(10,2),
radius DECIMAL(10,4),
mass DECIMAL(10,4),
distance_from_earth DECIMAL(12,4)
);
```

- **Primary Key:** `star_id` uniquely identifies each star. `AUTO_INCREMENT` means the database automatically generates the next number.
- **Data Types:** We use `DECIMAL(precision, scale)` for precise scientific measurements to avoid floating-point rounding errors.

```
CREATE TABLE exoplanets (
planet_id INT AUTO_INCREMENT PRIMARY KEY,
star_id INT,
name VARCHAR(100) NOT NULL,
-- ... other columns (radius, mass, etc.) ...
predicted_habitability DECIMAL(5,4),
last_updated DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
FOREIGN KEY (star_id) REFERENCES stars(star_id) ON DELETE SET NULL
);
```

- **Foreign Key:** `star_id` creates a **One-to-Many Relationship** (One star has many planets). It enforces **Referential Integrity**—you cannot add a planet to a star that doesn't exist.
- **ON DELETE SET NULL:** If a star is deleted, the database won't delete the planets; instead, it sets their `star_id` to NULL, protecting our planetary data.
- **Automatic Timestamps:** `ON UPDATE CURRENT_TIMESTAMP` tells MySQL to automatically update this column whenever the row is modified, ensuring accurate auditing.

3. Advanced SQL: Triggers, Views, and Procedures

3.1 Triggers (The "Confidence Drift" Engine)

A **Trigger** is a set of SQL statements that automatically "fires" when an event (INSERT, UPDATE, DELETE) occurs. We use them to build an audit trail of habitability scores without writing extra Python code.

```
CREATE TRIGGER after_exoplanet_update
AFTER UPDATE ON exoplanets
FOR EACH ROW
BEGIN
IF NEW.predicted_habitability <> OLD.predicted_habitability THEN
INSERT INTO habitability_log (planet_id, predicted_habitability, prediction_timestamp)
VALUES (NEW.planet_id, NEW.predicted_habitability, NOW());
END IF;
END$$
```

- **Explanation:** `AFTER UPDATE` means this runs immediately after a row in `exoplanets` changes.
- `NEW` **and** `OLD` **Keywords:** `OLD` holds the data *before* the update; `NEW` holds the data *after*. The `IF` statement checks if the habitability score actually changed. If it did, it inserts a new row into `habitability_log`. This is the core of our **Novelty Feature**—tracking how predictions drift over time as scientists update planet measurements.

3.2 Views

A **View** is a virtual table based on the result-set of an SQL statement. It abstracts complex queries so the backend can query it like a normal table.

```
CREATE VIEW v_current_habitability AS
SELECT e.*, hl.prediction_timestamp AS last_prediction_time
FROM exoplanets e
JOIN habitability_log hl ON e.planet_id = hl.planet_id
WHERE hl.log_id = (
SELECT MAX(log_id) FROM habitability_log WHERE planet_id = e.planet_id
);
```

- **Explanation:** This view joins the `exoplanets` table with the `habitability_log` table.
- **Subquery:** The `WHERE hl.log_id = (SELECT MAX...)` is a correlated subquery that ensures we only grab the *most recent* log entry for each planet, filtering out the historical noise.

3.3 Stored Procedures

A **Stored Procedure** is prepared SQL code that you can save and reuse. It executes on the database server, which is faster than sending multiple queries from Python.

```
CREATE PROCEDURE GetDriftHistory(IN p_id INT)
BEGIN
SELECT predicted_habitability, prediction_timestamp
FROM habitability_log
WHERE planet_id = p_id
ORDER BY prediction_timestamp ASC;
END$$
```

- **Explanation:** This procedure takes an input parameter (`IN p_id`). The backend calls `CALL GetDriftHistory(12)` to instantly get the chronological history of a planet's score changes.

4. Machine Learning & Methodology

The Algorithm: Random Forest Classifier

To predict habitability, we trained a Machine Learning model. We chose **Random Forest**, an *Ensemble Learning* method.

- **How it works:** It builds hundreds of mathematical "Decision Trees" during training. Each tree looks at features (like mass, temp, radius) and votes on whether the planet is habitable. The Forest takes the majority vote.
- **Why we used it:** Astronomical data is noisy and non-linear. Random Forests handle non-linear relationships excellently and are highly resistant to overfitting compared to single decision trees.

Feature Engineering: Earth Similarity Index (ESI)

Before training, we calculated the ESI for each planet. The ESI formula compares a planet to Earth (where Earth = 1.0).

`ESI = 1 - 0.5 * (	x - x0	/	x + x0	) where x is a planetary property (like radius) and x0` is Earth's value.

By feeding this engineered feature into the Random Forest, we significantly improved the model's accuracy.

5. The Backend (Python, Flask, REST API)

The backend acts as the bridge. We use **Flask** to create a **REST API** (Representational State Transfer). REST relies on standard HTTP methods:

- `GET` (Read data)
- `POST` (Create data)
- `PUT` (Update data)
- `DELETE` (Remove data)

Code Walkthrough: Creating a Planet (`backend/routes/exoplanets.py`)

```
@exoplanets_bp.route('/', methods=['POST'])
def create_exoplanet():
data = request.json

# 1. Run the Machine Learning Model
score = get_prediction(data)

# 2. Connect to Database
conn = get_db_connection()
cursor = conn.cursor()

# 3. Execute Parameterized SQL
cursor.execute("""
INSERT INTO exoplanets (star_id, name, radius, predicted_habitability)
VALUES (%s, %s, %s, %s)
""", (data.get('star_id'), data.get('name'), data.get('radius'), score))

# 4. Commit and Close
conn.commit()
conn.close()

return jsonify({"message": "Exoplanet created"})
```

- `request.json`**:** Grabs the incoming data from the frontend.
- `get_prediction()`**:** This function loads our `model.pkl` file, feeds it the new planet's data, and gets the AI's habitability prediction (e.g., 0.85).
- **Parameterized Query (`%s`):** We use `%s` placeholders instead of directly injecting variables into the SQL string. This is a critical security practice to prevent **SQL Injection Attacks**.

- `conn.commit()`**:** Databases use **Transactions**. `commit()` tells MySQL to permanently save the changes. If an error occurred, we would use `rollback()` to undo it, preserving database **ACID properties**.

6. The Frontend (HTML, CSS, Asynchronous JavaScript)

The UI is built with a modern, dark "Space Mission Control" theme utilizing **Glassmorphism** (CSS `backdrop-filter: blur()`).

Asynchronous JavaScript (AJAX / Fetch)

In legacy web apps, submitting a form reloads the entire webpage. We use modern Asynchronous JavaScript (`fetch`) to create a **Single Page Application (SPA)** feel.

```
async function loadPlanets() {
// 1. Make network request to Python backend
const res = await fetch('http://localhost:5000/api/exoplanets');
const planets = await res.json();

// 2. Manipulate the DOM
for (let p of planets) {
const tr = document.createElement('tr');
tr.innerHTML = `<td>${p.name}</td> <td>${p.predicted_habitability}</td>`;
tbody.appendChild(tr);
}
}
```

- `async / await`**:** Network requests take time. `await` tells JavaScript to pause this specific function until the Python server responds, without freezing the user's browser.
- **DOM Manipulation:** Once the data (JSON) arrives, JavaScript loops through it and dynamically creates HTML (table row) elements, injecting them into the webpage instantly.

7. The Full Execution Flow (Putting it all together)

Let's trace exactly what happens when you add a planet in the UI:

1. **User Input:** You type "Kepler-22b", Radius "2.4", Mass "36", into the HTML form and click Save.
2. **Frontend Event:** JavaScript intercepts the click, packages the data into a JSON object, and sends an HTTP `POST` request to `http://localhost:5000/api/exoplanets`.
3. **Backend Routing:** Flask receives the URL, matches it to the `@exoplanets_bp.route('/', methods=['POST'])` function.
4. **Machine Learning:** The Python code loads the Random Forest model, passes in Radius 2.4 and Mass 36, and calculates a predicted habitability score of `0.65`.
5. **Database Insertion:** Python executes the `INSERT INTO exoplanets` query, passing the user data *and* the `0.65` score.
6. **Database Trigger:** The moment the row hits the MySQL `exoplanets` table, the `AFTER INSERT` trigger wakes up and copies the planet ID, the `0.65` score, and the current clock time into the `habitability_log` table.
7. **Response:** MySQL tells Python "Success". Python tells JavaScript "Success". JavaScript clears the form and re-runs `loadPlanets()` to show the new data on the screen.

8. Novelty Explanation: The Confidence Drift Alert

The "Confidence Drift" feature is the culmination of all three layers working together.

1. The **Database** tracks historical scores via Triggers.
2. The **Backend** exposes this history via the `GetDriftHistory` Stored Procedure.
3. The **Frontend** compares the *latest* score to the *previous* score.

If the absolute difference is greater than 15% (e.g., `Math.abs(latest - prev) > 0.15`), JavaScript dynamically renders a neon orange warning badge: ■ `Drift: increased from 0.40 to 0.65`. This visually alerts researchers that recent data corrections have drastically altered the AI's confidence in the planet's ability to support life.